Lecture 5:

Synchronization

CS211 - Operating systems

Spring 2026

Independent vs cooperating threads

	Independent threads	Cooperating threads

Independent vs cooperating threads

	Independent threads	Cooperating threads
Shared states		

Independent vs cooperating threads

	Independent threads	Cooperating threads
Shared states	No	Yes

Independent vs cooperating threads

	Independent threads	Cooperating threads
Shared states	No	Yes
Example		

Independent vs cooperating threads

	Independent threads	Cooperating threads
Shared states	No	Yes
Example	Thread 1: x = 1 Thread 2: y = 2	

Independent vs cooperating threads

	Independent threads	Cooperating threads
Shared states	No	Yes
Example	Thread 1: x = 1 Thread 2: y = 2	Thread 1: x = y + 1 Thread 2: y = y*2

Independent vs cooperating threads

	Independent threads	Cooperating threads
Shared states	No	Yes
Example	Thread 1: x = 1 Thread 2: y = 2	Thread 1: x = y + 1 Thread 2: y = y*2

Independent vs cooperating threads

	Independent threads	Cooperating threads
Shared states	No	Yes
Example	Thread 1: x = 1 Thread 2: y = 2	Thread 1: x = y + 1 Thread 2: y = y*2

Independent vs cooperating threads

	Independent threads	Cooperating threads
Shared states	No	Yes
Example	Thread 1: x = 1 Thread 2: y = 2	Thread 1: x = y + 1 Thread 2: y = y*2

Independent vs cooperating threads

	Independent threads	Cooperating threads
Shared states	No	Yes
Example	Thread 1: x = 1 Thread 2: y = 2	Thread 1: x = y + 1 Thread 2: y = y*2
Behavior		

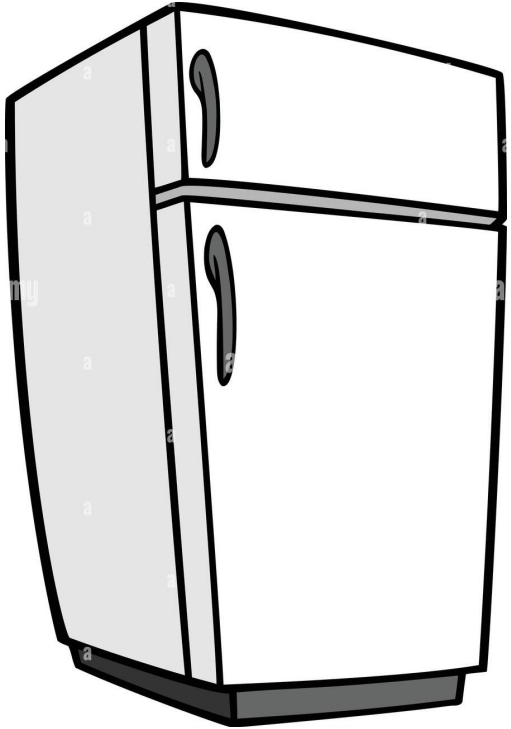
Independent vs cooperating threads

	Independent threads	Cooperating threads
Shared states	No	Yes
Example	Thread 1: x = 1 Thread 2: y = 2	Thread 1: x = y + 1 Thread 2: y = y*2
Behavior	Deterministic Reproducible	

Independent vs cooperating threads

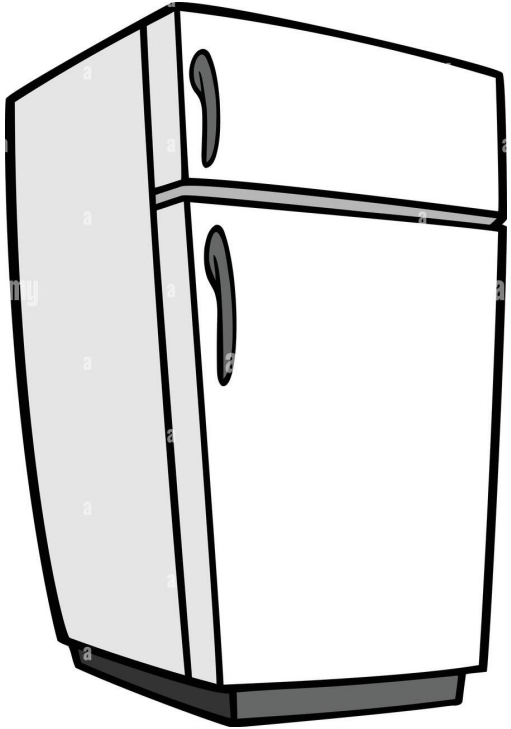
	Independent threads	Cooperating threads
Shared states	No	Yes
Example	Thread 1: x = 1 Thread 2: y = 2	Thread 1: x = y + 1 Thread 2: y = y*2
Behavior	Deterministic Reproducible	Nondeterministic May be irreproducible

Example



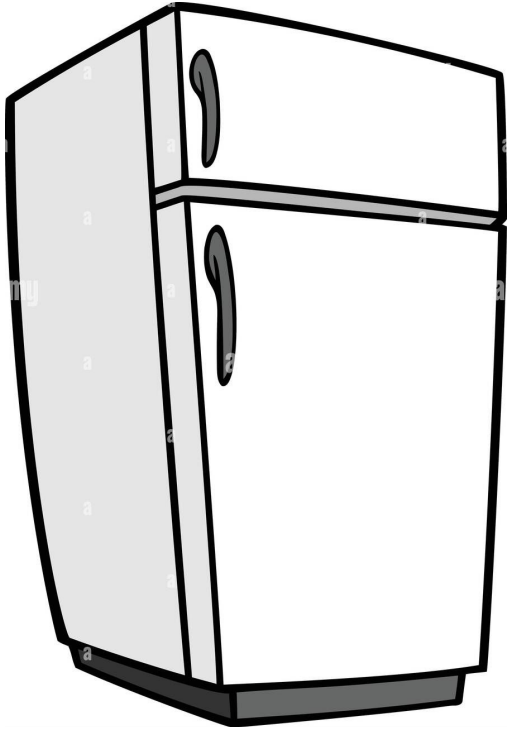
Example

	You (A)
3:00	Look in Fridge. Out of milk
3:05	Leave for store
3:10	Arrive at store
3:15	Buy milk
3:20	Arrive home, put milk away
3:25	
3:10	



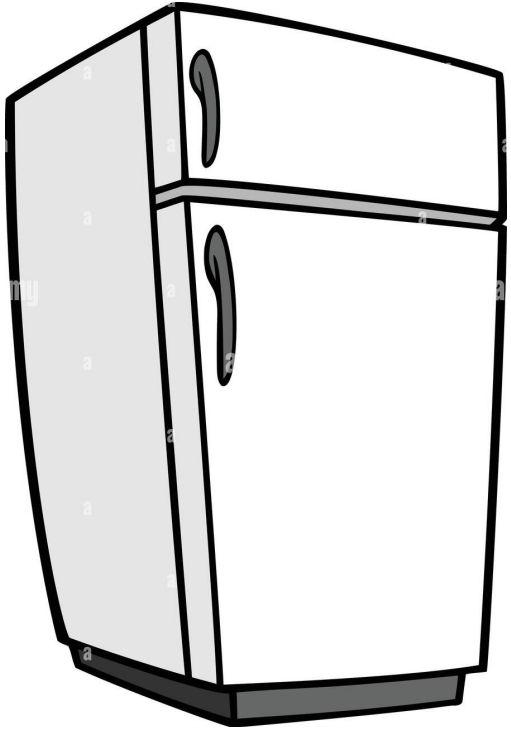
Example

	You (A)	Your roommate (B)
3:00	Look in Fridge. Out of milk	
3:05	Leave for store	
3:10	Arrive at store	Look in Fridge. Out of milk
3:15	Buy milk	Leave for store
3:20	Arrive home, put milk away	Arrive at store
3:25		Buy milk
3:10		Arrive home, put milk away



Example

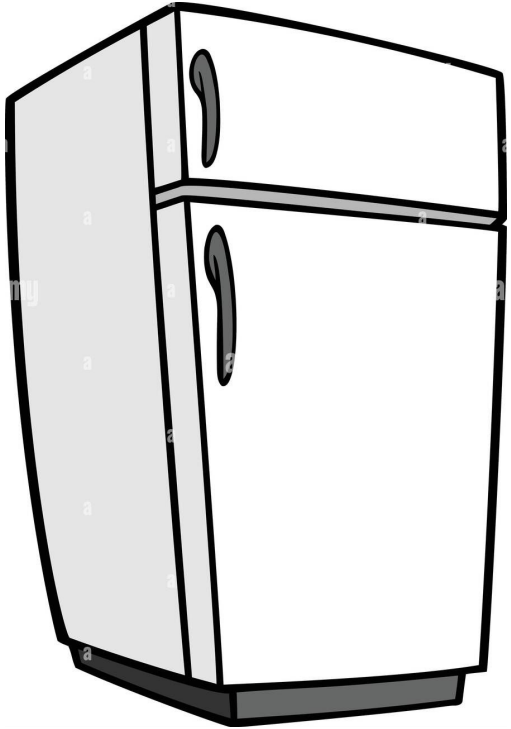
	You (A)	Your roommate (B)
3:00	Look in Fridge. Out of milk	
3:05	Leave for store	
3:10	Arrive at store	Look in Fridge. Out of milk
3:15	Buy milk	Leave for store
3:20	Arrive home, put milk away	Arrive at store
3:25		Buy milk
3:10		Arrive home, put milk away



"Too much milk" problem

Example

	You (A)	Your roommate (B)
3:00	Look in Fridge. Out of milk	
3:05	Leave for store	
3:10	Arrive at store	Look in Fridge. Out of milk
3:15	Buy milk	Leave for store
3:20	Arrive home, put milk away	Arrive at store
3:25		Buy milk
3:10		Arrive home, put milk away

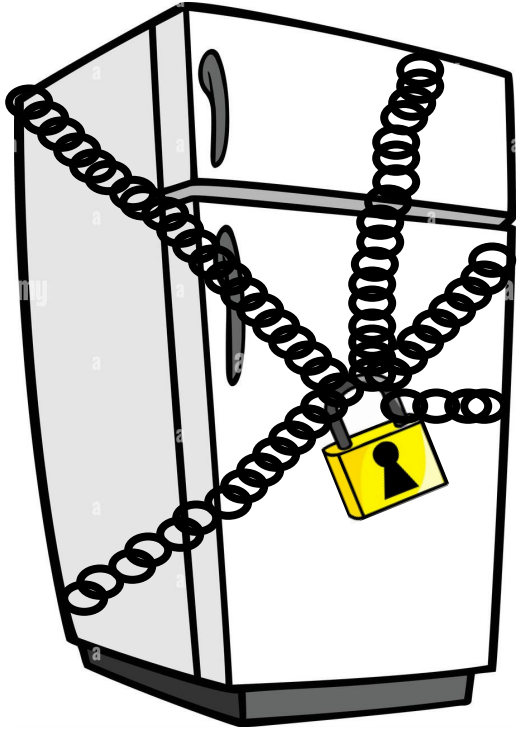


"Too much milk" problem

How to avoid?

Example

	You (A)	Your roommate (B)
3:00	Look in Fridge. Out of milk	
3:05	Leave for store	
3:10	Arrive at store	Look in Fridge. Out of milk
3:15	Buy milk	Leave for store
3:20	Arrive home, put milk away	Arrive at store
3:25		Buy milk
3:10		Arrive home, put milk away

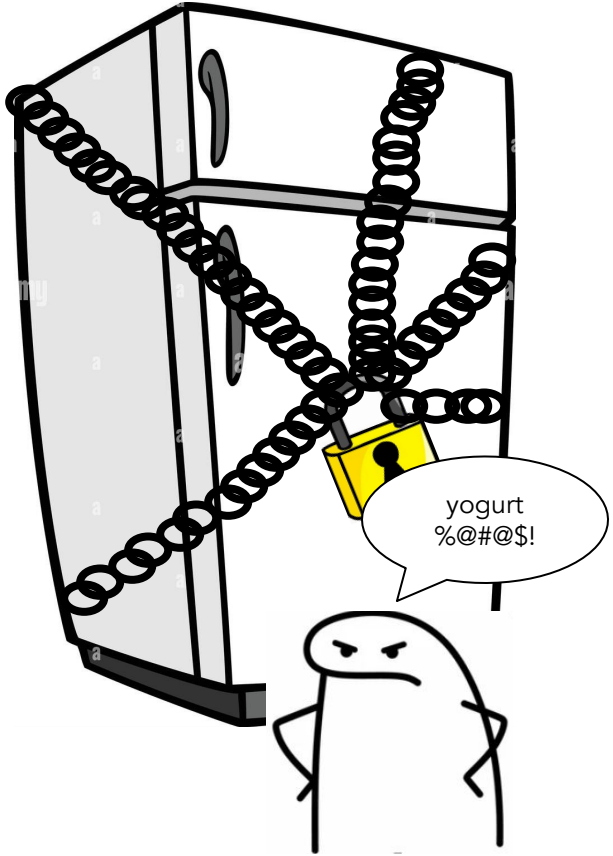


"Too much milk" problem

How to avoid?

Example

	You (A)	Your roommate (B)
3:00	Look in Fridge. Out of milk	
3:05	Leave for store	
3:10	Arrive at store	Look in Fridge. Out of milk
3:15	Buy milk	Leave for store
3:20	Arrive home, put milk away	Arrive at store
3:25		Buy milk
3:10		Arrive home, put milk away

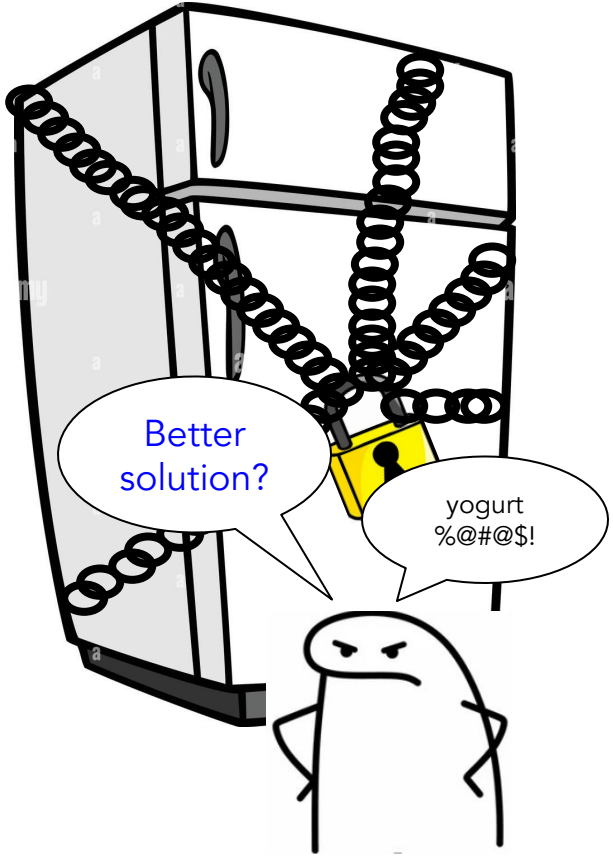


"Too much milk" problem

How to avoid?

Example

	You (A)	Your roommate (B)
3:00	Look in Fridge. Out of milk	
3:05	Leave for store	
3:10	Arrive at store	Look in Fridge. Out of milk
3:15	Buy milk	Leave for store
3:20	Arrive home, put milk away	Arrive at store
3:25		Buy milk
3:10		Arrive home, put milk away



"Too much milk" problem

How to avoid?

Example

	You (A)	Your roommate (B)
3:00	Look in Fridge. Out of milk	
3:05	Leave for store	
3:10	Arrive at store	Look in Fridge. Out of milk
3:15	Buy milk	Leave for store
3:20	Arrive home, put milk away	Arrive at store
3:25		Buy milk
3:10		Arrive home, put milk away

"Too much milk" problem

How to avoid?

Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```

Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```

<u>Thread A</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```

<u>Thread A</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```

<u>Thread A</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

<u>Thread B</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```

<u>Thread A</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

<u>Thread B</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```

<u>Thread A</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

<u>Thread B</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```

<u>Thread A</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

<u>Thread B</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```

<u>Thread A</u>

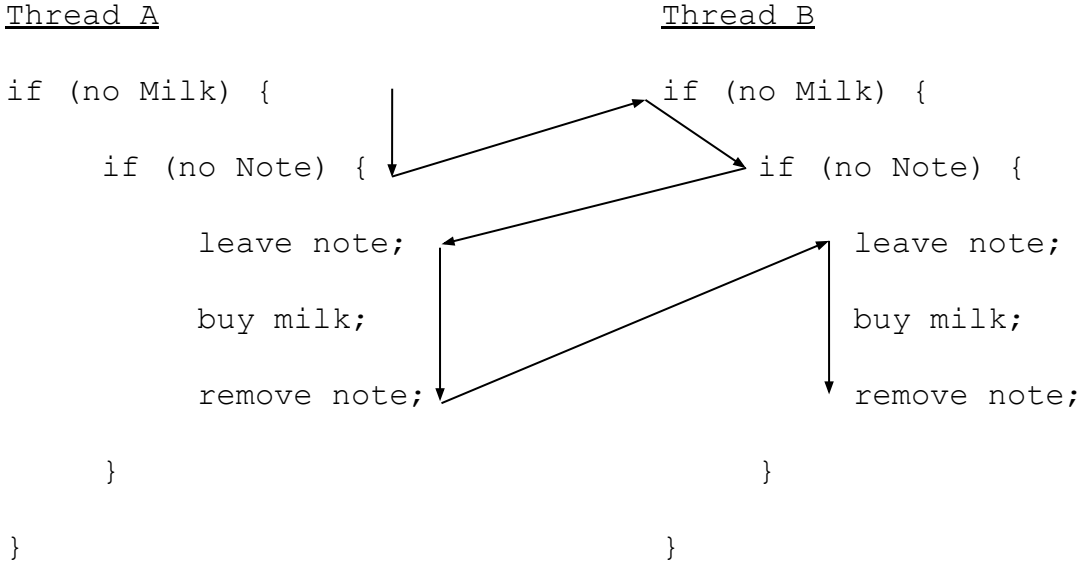
```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

<u>Thread B</u>

```
if (no Milk) {

     if (no Note) {

          leave note;

          buy milk;

          remove note;

     }

}
```

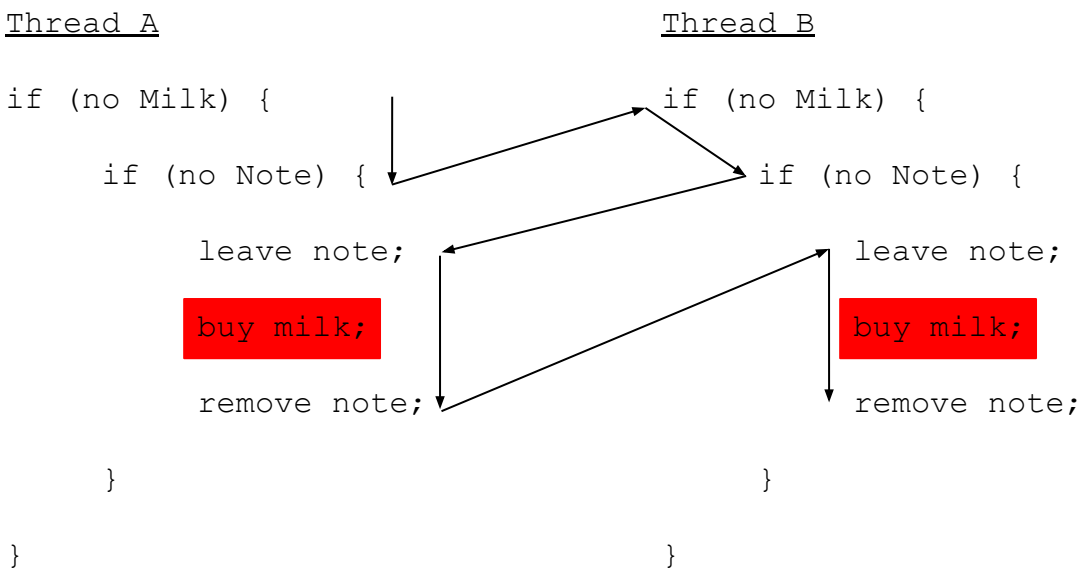
Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```



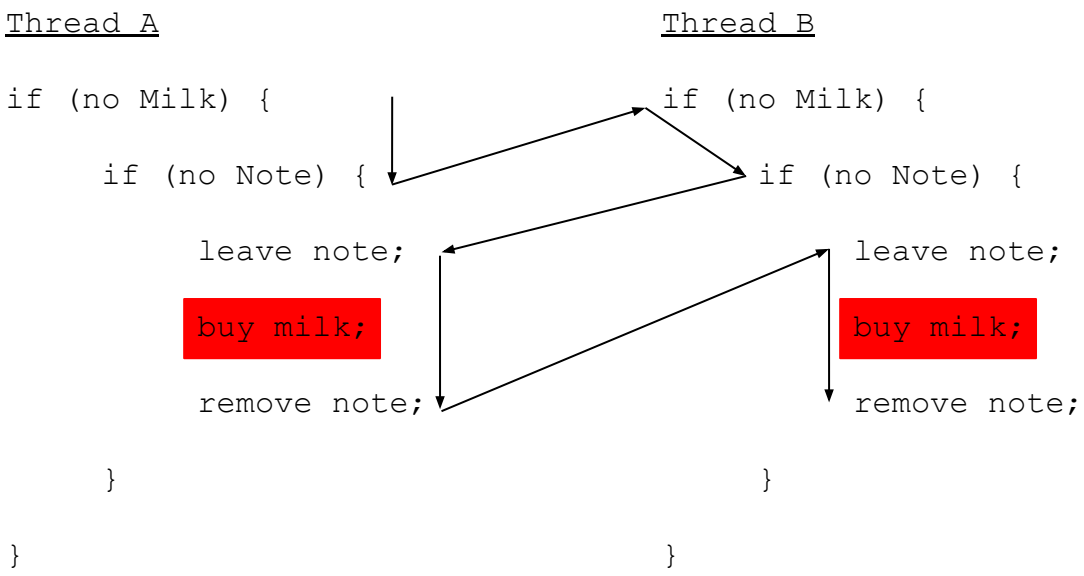
Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```



Solution 1

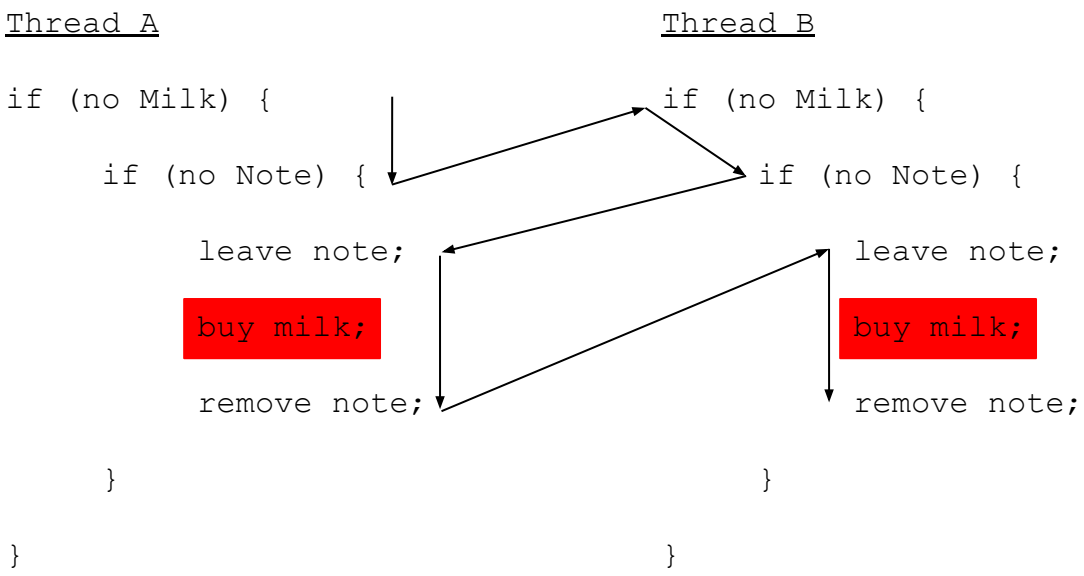
```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```



- Still “Too much milk”

Solution 1

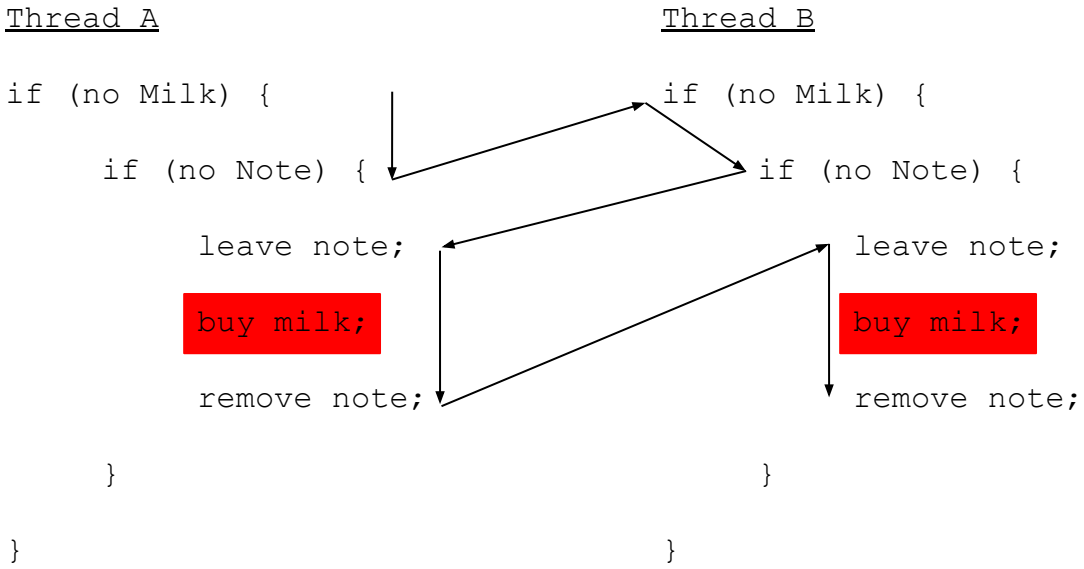
```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```



- Still "Too much milk"
  - But only occasionally

Solution 1

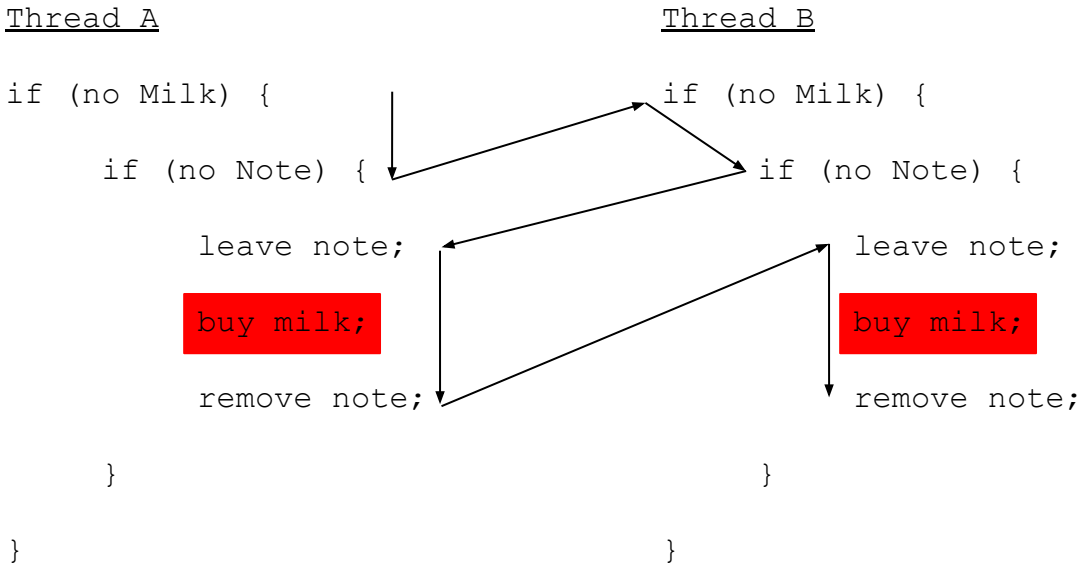
```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```



- Still "Too much milk"
  - But only occasionally
    - Is it better?

Solution 1

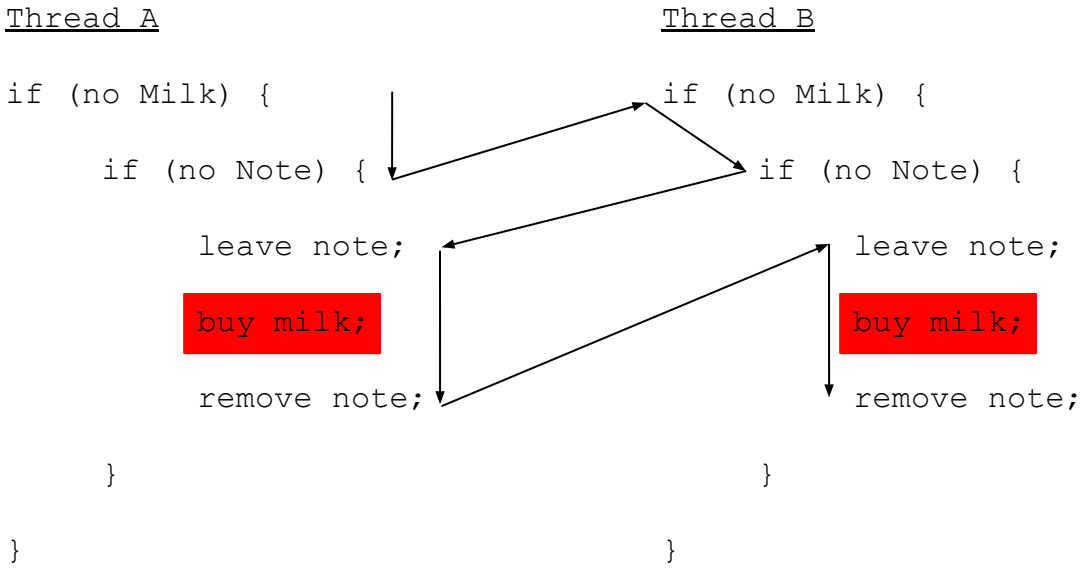
```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```



- Still "Too much milk"
  - But only occasionally
    - Is it better?
      - No, it is even worse since the program fails intermittently

Solution 1

```
if (no Milk) {
     if (no Note) {
          leave note;
          buy milk;
          remove note;
     }
}
```



- Still "Too much milk"
  - But only occasionally
    - Is it better?
      - No, it is even worse since the program fails intermittently
        - Hard to debug
        - The program must work in all scenarios

Solution 2: Leaving a note first

Solution 2: Leaving a note first

```
➢   leave note;
    if (no Milk) {
         if (no Note) {
              buy milk;
         }
    }
    remove note;
```

Solution 2: Leaving a note first

```
➢   leave note;
    if (no Milk) {
         if (no Note) {
              buy milk;
         }
    }
    remove note;
```

Solution 2: Leaving a note first

```
➢   leave note;
    if (no Milk) {
        if (no Note) {
            buy milk;
        }
    }
    remove note;
```

Never buy milk!

Solution 3: Using two notes

Solution 3: Using two notes

- ```
  Thread A
  leave note A;
  if (no Note B) {
       if (no Milk) {
            buy Milk;
       }
  }
  remove note A;
  ```

- ```
  Thread B
  leave note B;
  if (no Note A) {
       if (no Milk) {
            buy Milk;
       }
  }
  remove note B;
  ```

Solution 3: Using two notes

- Thread A

```
leave note A;
if (no Note B) {
     if (no Milk) {
          buy Milk;
     }
}
remove note A;
```

- Thread B

```
leave note B;
if (no Note A) {
     if (no Milk) {
          buy Milk;
     }
}
remove note B;
```

Solution 3: Using two notes

```
Thread A
leave note A;
if (no Note B) {
     if (no Milk) {
          buy Milk;
     }
}
remove note A;
```

```
Thread B
leave note B;
if (no Note A) {
     if (no Milk) {
          buy Milk;
     }
}
remove note B;
```

Solution 3: Using two notes

```
Thread A
leave note A;
if (no Note B) {
     if (no Milk) {
          buy Milk;
     }
}
remove note A;
```

```
Thread B
leave note B;
if (no Note A) {
     if (no Milk) {
          buy Milk;
     }
}
remove note B;
```

Solution 3: Using two notes

- Thread A

```
leave note A;
if (no Note B) {
     if (no Milk) {
          buy Milk;
     }
}
remove note A;
```

- Thread B

```
leave note B;
if (no Note A) {
     if (no Milk) {
          buy Milk;
     }
}
remove note B;
```

Solution 3: Using two notes

- Thread A

```
leave note A;
if (no Note B) {
      if (no Milk) {
            buy Milk;
      }
}
remove note A;
```

- Thread B

```
leave note B;
if (no Note A) {
      if (no Milk) {
            buy Milk;
      }
}
remove note B;
```

Solution 3: Using two notes

- Thread A

```
leave note A;
if (no Note B) {
      if (no Milk) {
            buy Milk;
      }
}
remove note A;
```

- Thread B

```
leave note B;
if (no Note A) {
      if (no Milk) {
            buy Milk;
      }
}
remove note B;
```

Solution 3: Using two notes

- Thread A

```
leave note A;
if (no Note B) {
      if (no Milk) {
            buy Milk;
      }
}
remove note A;
```

- Thread B

```
leave note B;
if (no Note A) {
      if (no Milk) {
            buy Milk;
      }
}
remove note B;
```

Solution 3: Using two notes

- Thread A

```
leave note A;
if (no Note B) {
      if (no Milk) {
            buy Milk;
      }
}
remove note A;
```

- Thread B

```
leave note B;
if (no Note A) {
      if (no Milk) {
            buy Milk;
      }
}
remove note B;
```

Solution 3: Using two notes

- Thread A

```
leave note A;
if (no Note B) {
      if (no Milk) {
            buy Milk;
      }
}
remove note A;
```

- Thread B

```
leave note B;
if (no Note A) {
      if (no Milk) {
            buy Milk;
      }
}
remove note B;
```

- No one buys milk!

Solution 3: Using two notes

- Thread A

```
leave note A;
if (no Note B) {
      if (no Milk) {
            buy Milk;
      }
}
remove note A;
```

- Thread B

```
leave note B;
if (no Note A) {
      if (no Milk) {
            buy Milk;
      }
}
remove note B;
```

- No one buys milk!
  - but extremely rare

Solution 3: Using two notes

- Thread A

```
leave note A;
if (no Note B) {
      if (no Milk) {
            buy Milk;
      }
}
remove note A;
```

- Thread B

```
leave note B;
if (no Note A) {
      if (no Milk) {
            buy Milk;
      }
}
remove note B;
```

- No one buys milk!
  - but extremely rare
- Probably something like this in UNIX

Solution 4

- ```
  Thread A
  leave note A;
  if (no Note B) {
       if (no Milk) {
            buy milk;
       }
  }
  remove note A;
  ```

- ```
  Thread B
  leave note B;
  while (note A) {
       do nothing;
  }
  if (no Milk) {
       buy milk;
  }
  remove note B;
  ```

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
     if (no Milk) {
          buy milk;
     }
}
remove note A;
```

- Thread B

```
leave note B;
while (note A) {
     do nothing;
}
if (no Milk) {
     buy milk;
}
remove note B;
```

- Is this solution correct?

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

- Is this solution correct?

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

executed before

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

- Is this solution correct?
  - Case 1

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

executed before

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

- Is this solution correct?
  - Case 1

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

executed before

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

- Is this solution correct?
  - Case 1

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

executed before

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

- Is this solution correct?
  - Case 1: “if (no Note B)” is executed before “leave note B” → Thread A will buy milk

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

executed before

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

- Is this solution correct?
  - Case 1: “if (no Note B)” is executed before “leave note B” → Thread A will buy milk
  - Case 2

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

- Is this solution correct?
  - Case 1: “if (no Note B)” is executed before “leave note B” → Thread A will buy milk
  - Case 2

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

executed before

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

- Is this solution correct?
  - Case 1: “if (no Note B)” is executed before “leave note B” → Thread A will buy milk
  - Case 2

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

executed before

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

- Is this solution correct?
  - Case 1: “if (no Note B)” is executed before “leave note B” → Thread A will buy milk
  - Case 2

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

executed before

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

This will be finally executed when there is no note A

- Is this solution correct?
  - Case 1: "if (no Note B)" is executed before "leave note B" → Thread A will buy milk
  - Case 2

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
    if (no Milk) {
        buy milk;
    }
}
remove note A;
```

executed before

- Thread B

```
leave note B;
while (note A) {
    do nothing;
}
if (no Milk) {
    buy milk;
}
remove note B;
```

This will be finally executed when there is no note A

- Is this solution correct?
  - Case 1: "if (no Note B)" is executed before "leave note B" → Thread A will buy milk
  - Case 2: "if (no Note B)" is executed after "leave note B" → Thread B will buy milk

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
     if (no Milk) {
          buy milk;
     }
}
remove note A;
```

- Thread B

```
leave note B;
while (note A) {
     do nothing;
}
if (no Milk) {
     buy milk;
}
remove note B;
```

- Is this solution correct? Yes

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
     if (no Milk) {
          buy milk;
     }
}
remove note A;
```

- Thread B

```
leave note B;
while (note A) {
     do nothing;
}
if (no Milk) {
     buy milk;
}
remove note B;
```

- Is this solution correct? Yes
- Is this solution good?

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
     if (no Milk) {
          buy milk;
     }
}
remove note A;
```

- Thread B

```
leave note B;
while (note A) {
     do nothing;
}
if (no Milk) {
     buy milk;
}
remove note B;
```

- Is this solution correct? Yes
- Is this solution good? No

Solution 4

- ```
  Thread A
  leave note A;
  if (no Note B) {
      if (no Milk) {
          buy milk;
      }
  }
  remove note A;
  ```

- ```
  Thread B
  leave note B;
  while (note A) {
      do nothing;
  }
  if (no Milk) {
      buy milk;
  }
  remove note B;
  ```

- Is this solution correct? Yes
- Is this solution good? No
  - Hard to prove

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
     if (no Milk) {
          buy milk;
     }
}
remove note A;
```

- Thread B

```
leave note B;
while (note A) {
     do nothing;
}
if (no Milk) {
     buy milk;
}
remove note B;
```

- Is this solution correct? Yes
- Is this solution good? No
  - Hard to prove
  - Different code for each thread

Solution 4

- Thread A

```
leave note A;
if (no Note B) {
     if (no Milk) {
          buy milk;
     }
}
remove note A;
```

- Thread B

```
leave note B;
while (note A) {
     do nothing;
}
if (no Milk) {
     buy milk;
}
remove note B;
```

busy waiting

- Is this solution correct? Yes
- Is this solution good? No
  - Hard to prove
  - Different code for each thread
  - "Busy waiting"

Concurrent programming

Concurrent programming

- We still can develop code to solve complex concurrent programming problems

Concurrent programming

- We still can develop code to solve complex concurrent programming problems

Computer Systems

G. Bell, D. Siewiorek, and S.H. Fuller, Editors

A New Solution of Dijkstra's Concurrent Programming Problem

Leslie Lamport
Massachusetts Computer Associates, Inc.

A simple solution to the mutual exclusion problem is

Concurrent programming

- We still can develop code to solve complex concurrent programming problems
- But it is not easy

Concurrent programming

- We still can develop code to solve complex concurrent programming problems
- But it is not easy
  - Code of popular tools still has concurrency bugs
    - Madanlal Musuvathi et al., 2008; Shan Lu et al., 2008; Sara Abbaspour Asadollah et al.,, 2017; Tengfei Tu et al., 2019

Concurrent programming

- We still can develop code to solve complex concurrent programming problems
- But it is not easy
  - Code of popular tools still has concurrency bugs
    - Madanlal Musuvathi et al., 2008; Shan Lu et al., 2008; Sara Abbaspour Asadollah et al.,, 2017; Tengfei Tu et al., 2019
- Need a high level methodology rather than ad-hoc solutions

Concurrent programming

- We still can develop code to solve complex concurrent programming problems
- But it is not easy
  - Code of popular tools still has concurrency bugs
    - Madanlal Musuvathi et al., 2008; Shan Lu et al., 2008; Sara Abbaspour Asadollah et al.,, 2017; Tengfei Tu et al., 2019
- Need a high level methodology rather than ad-hoc solutions

```
if (nomilk)
    buy milk;
```

Concurrent programming

- We still can develop code to solve complex concurrent programming problems
- But it is not easy
  - Code of popular tools still has concurrency bugs
    - Madanlal Musuvathi et al., 2008; Shan Lu et al., 2008; Sara Abbaspour Asadollah et al.,, 2017; Tengfei Tu et al., 2019
- Need a high level methodology rather than ad-hoc solutions

critical section

```
if (nomilk)
    buy milk;
```

Concurrent programming

- We still can develop code to solve complex concurrent programming problems
- But it is not easy
  - Code of popular tools still has concurrency bugs
    - Madanlal Musuvathi et al., 2008; Shan Lu et al., 2008; Sara Abbaspour Asadollah et al.,, 2017; Tengfei Tu et al., 2019
- Need a high level methodology rather than ad-hoc solutions

critical section

```
if (nomilk)
    buy milk;
```

Need to ensure only one thread is executed this code at one time

Concurrent programming

- We still can develop code to solve complex concurrent programming problems
- But it is not easy
  - Code of popular tools still has concurrency bugs
    - Madanlal Musuvathi et al., 2008; Shan Lu et al., 2008; Sara Abbaspour Asadollah et al.,, 2017; Tengfei Tu et al., 2019
- Need a high level methodology rather than ad-hoc solutions

critical section

```
if (nomilk)
    buy milk;
```

Need to ensure only one thread is executed this code at one time

HOW???

Concurrent programming

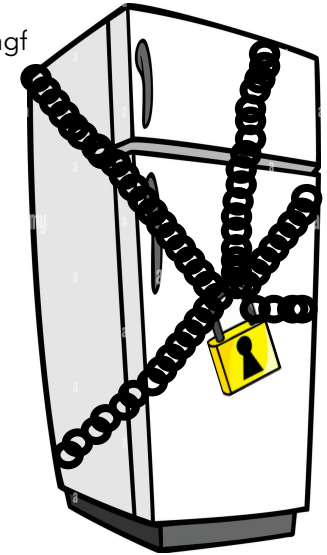
- We still can develop code to solve complex concurrent programming problems
- But it is not easy
  - Code of popular tools still has concurrency bugs
    - Madanlal Musuvathi et al., 2008; Shan Lu et al., 2008; Sara Abbaspour Asadollah et al.,, 2017; Tengf
- Need a high level methodology rather than ad-hoc solutions

critical section

```
if (nomilk)
    buy milk;
```

Need to ensure only one thread is executed this code at one time

HOW???



Concurrent programming

- Lock

critical section

```
if (nomilk)
    buy milk;
```

Need to ensure only one thread is executed this code at one time

HOW???

Concurrent programming

- Lock
  - acquire()

critical section

```
if (nomilk)
    buy milk;
```

Need to ensure only one thread is executed this code at one time

HOW???

Concurrent programming

- Lock
  - acquire()
    - wait until lock is free, then grab

critical section

```
if (nomilk)
    buy milk;
```

Need to ensure only one thread is executed this code at one time

HOW???

Concurrent programming

- Lock
  - acquire()
    - wait until lock is free, then grab
  - release()

critical section

```
if (nomilk)
    buy milk;
```

Need to ensure only one thread is executed this code at one time

HOW???

Concurrent programming

- Lock
  - acquire()
    - wait until lock is free, then grab
  - release()
    - Unlock, waking up anyone waiting

critical section

```
if (nomilk)
    buy milk;
```

Need to ensure only one thread is executed this code at one time

HOW???

Concurrent programming

- Lock
  - acquire()
    - wait until lock is free, then grab
  - release()
    - Unlock, waking up anyone waiting

```
milklock.acquire();
```

critical section

```
if (nomilk)
    buy milk;
```

Need to ensure only one thread is executed this code at one time

HOW???

Concurrent programming

- Lock
  - acquire()
    - wait until lock is free, then grab
  - release()
    - Unlock, waking up anyone waiting

```
milklock.acquire();
if (nomilk)
    buy milk;
milklock.release();
```

critical section

Need to ensure only one thread is executed this code at one time

HOW???

Concurrent programming

- Lock
  - acquire()
    - wait until lock is free, then grab
  - release()
    - Unlock, waking up anyone waiting
- But how to implement the lock???

```
milklock.acquire();
if (nomilk)
    buy milk;
milklock.release();
```

critical section

Need to ensure only one thread is executed this code at one time

HOW???

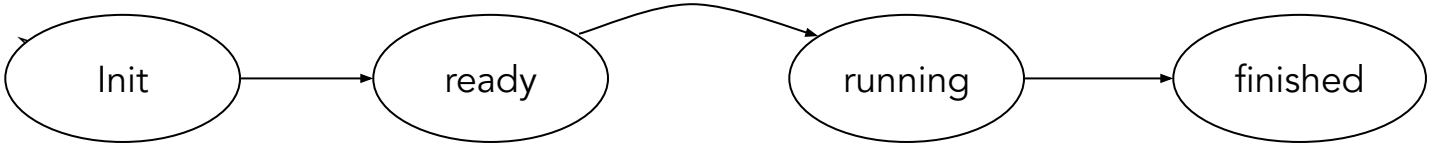
How to implement a lock?

How to implement a lock?

- How to ensure that only one thread is executed?

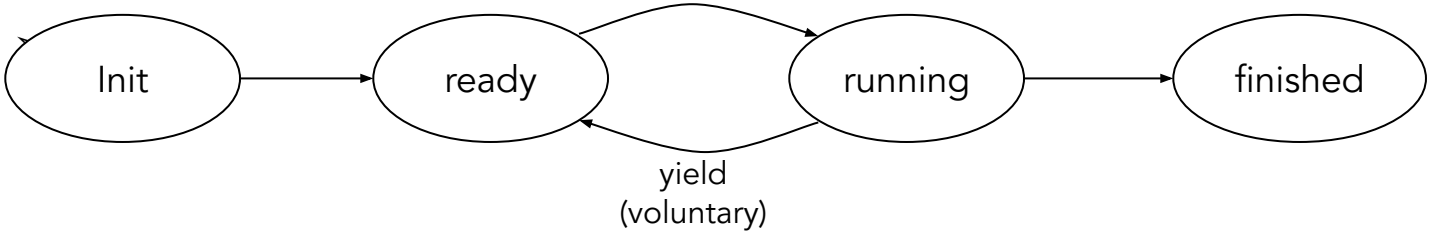
How to implement a lock?

- How to ensure that only one thread is executed?



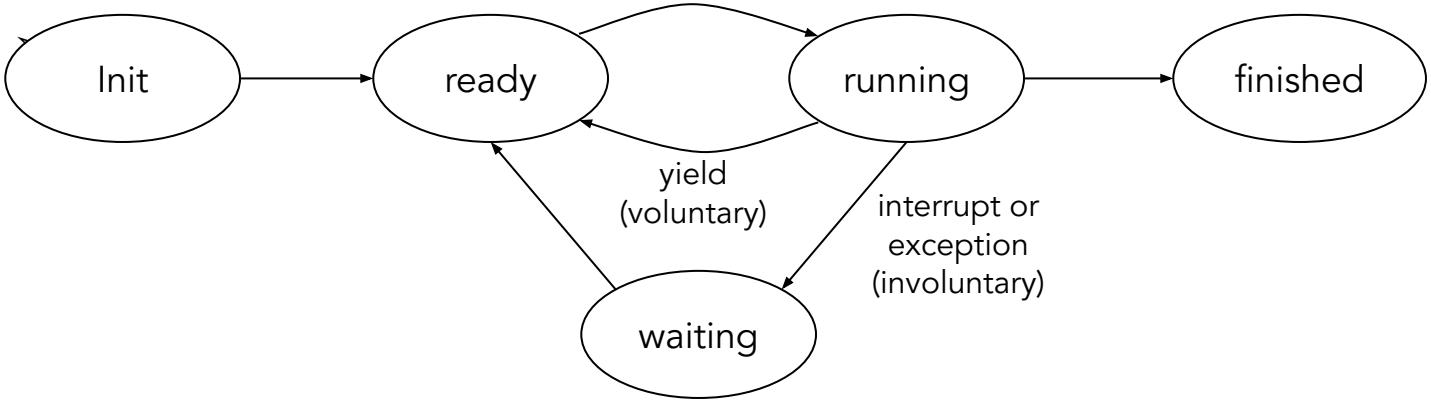
How to implement a lock?

- How to ensure that only one thread is executed?



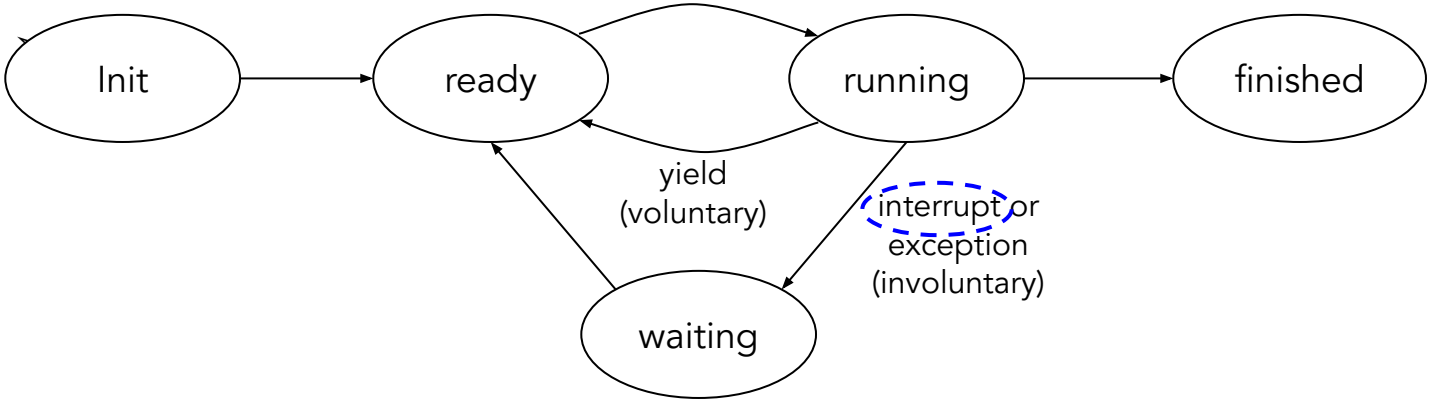
How to implement a lock?

- How to ensure that only one thread is executed?



How to implement a lock?

- How to ensure that only one thread is executed?



How to implement a lock?

- Disable interrupts

How to implement a lock?

- Disable interrupts
  - Naive implementation (version 1.0)

```
acquire() { disable interrupts; }
release() { enable interrupts; }
```

How to implement a lock?

- Disable interrupts
  - Naive implementation (version 1.0)

```
acquire() { disable interrupts; }
release() { enable interrupts; }
```

Is it good?

How to implement a lock?

- Disable interrupts
  - Naive implementation (version 1.0)

```
acquire() { disable interrupts; }
release() { enable interrupts; }
```

Is it good?

What happens with this?

```
acquire();
while(true) {};
release();
```

How to implement a lock?

- Disable interrupts
  - Naive implementation (version 1.0)

```
acquire() { disable interrupts; }
release() { enable interrupts; }
```

  - Better implementation (version 2.0)

How to implement a lock?

- Disable interrupts
  - Naive implementation (version 1.0)

```
acquire() { disable interrupts; }
release() { enable interrupts; }
```

  - Better implementation (version 2.0)

```
lock = FREE; //initially
```

How to implement a lock?

- Disable interrupts
  - Naive implementation (version 1.0)

```
acquire() { disable interrupts; }
release() { enable interrupts; }
```

  - Better implementation (version 2.0)

```
lock = FREE; //initially

acquire() {
    disable interrupts;
    if (lock == BUSY) {
        put thread on wait queue;
        go to sleep();
    } else {
        lock = BUSY;
    }
    enable interrupts;
}
```

How to implement a lock?

- Disable interrupts
  - Naive implementation (version 1.0)

```
acquire() { disable interrupts; }
release() { enable interrupts; }
```

  - Better implementation (version 2.0)

```
lock = FREE; //initially
```

```
acquire() {
    disable interrupts;
    if (lock == BUSY) {
        put thread on wait queue;
        go to sleep();
    } else {
        lock = BUSY;
    }
    enable interrupts;
}
```

```
release() {
    disable interrupts;
    if (anyone on wait queue) {
        take one thread off wait queue;
        place this thread on ready queue;
    } else {
        lock = FREE;
    }
    enable interrupts;
}
```

Atomic Read-Modify-Write Instructions

- Implementing a lock by disabling interrupts:
  - Can’t give the lock implementation to users
  - Doesn’t work well on multiprocessor

Atomic Read-Modify-Write Instructions

- Implementing a lock by disabling interrupts:
  - Can’t give the lock implementation to users
  - Doesn’t work well on multiprocessor
- Alternative: atomic instruction sequences
  - Reading and then writing a variable atomically
  - Implemented by the hardware
  - Can be used on both uniprocessors and multiprocessors

Examples of Read-Modify-Write

```
➢   test&set(&address) {                    /* most architectures */
          result = M[address];              // return result from "address" and
          M[address] = 1;                   // set value at "address" to 1
          return result;
    }
➢   swap(&address, register) {              /* x86 */
          temp = M[address];                // swap register's value to
          M[address] = register;            // value at "address"
          register = temp;                  // value from "address" put back to register
          return temp;                      // value from "address" considered return from swap
    }
➢   compare&swap(&address, reg1, reg2) {/* x86 (returns old value), 68000 */
          if (reg1 == M[address]) {         // If memory still == reg1,
                M[address] = reg2;          // then put reg2 => memory
                return success;
          } else {                          // Otherwise do not change memory
                return failure;
          }
    }
➢   load-linked&store-conditional(&address) {/* R4000, alpha */
          Loop:
                ll r1, M[address];
                movi r2, 1;       // Can do arbitrary computation
                sc r2, M[address];
                beqz r2, loop;
    }
```

Implementing locks with test&set

- Does not require entry to the kernel (version 3.0, “spin lock”)

```
// Can access this memory location from user space!
int lock = FREE; // FREE = 0

acquire(int* lock) {
    while (test&set(lock));    // atomic operation!
}




release(int* lock) {
    *lock = FREE;              // atomic operation!
}
```

Implementing locks with test&set

- Does not require entry to the kernel (version 3.0, "spin lock")

```
// Can access this memory location from user space!
int lock = FREE;

acquire(int* lock) {
  while (test&set(lock));  // atomic operation!
}
```

busy waiting!

```
release(int* lock) {
    *lock = FREE;            // atomic operation!
}
```

Implementing locks with test&set

- Does not require entry to the kernel (version ~~3.0~~ 3.1, "spin lock")

```
// Can access this memory location from user space!
int lock = FREE;

acquire(int* lock) {
  while (test&set(lock));  // atomic operation!
}
```

busy waiting!

```
release(int* lock) {
    *lock = FREE;            // atomic operation!
}
```

Implementing locks with test&set

- Does not require entry to the kernel (version ~~3.0~~ 3.1, "spin lock")

```
// Can access this memory location from user space!
int lock = FREE;

acquire(int* lock) {
    while (test&set(lock));    // atomic operation!
}




release(int* lock) {
    *lock = FREE;              // atomic operation!
}
```

Implementing locks with test&set

- Does not require entry to the kernel (version ~~3.0~~ 3.1, "spin lock")

```
// Can access this memory location from user space!
int lock = FREE;

acquire(int* lock) {
    while (test&set(lock));    // atomic operation!
    do {
      while(*lock);            // Wait until the lock might be free (quick check/test!)
    } while (test&set(lock)); // atomic operation!
}

release(int* lock) {
    *lock = FREE;              // atomic operation!
}
```

Implementing locks with test&set

- Does not require entry to the kernel (version ~~3.0~~ 3.1, "spin lock")

```
// Can access this memory location from user space!
int lock = FREE;

acquire(int* lock) {
    while (test&set(lock));    // atomic operation!
    do {
      while(*lock);            // Wait until the lock might be free (quick check/test!)
    } while (test&set(lock));  // atomic operation!
}

release(int* lock) {
    *lock = FREE;              // atomic operation!
}
```

better … but still busy waiting!

Implementing locks with test&set

- Better solution (version 4.0)

Implementing locks with test&set

- Better solution (version 4.0)

```
#define FREE 0
#define BUSY 1
int guard = FREE;
int lock = FREE;
```

Implementing locks with test&set

- Better solution (version 4.0)

```
#define FREE 0
#define BUSY 1
int guard = FREE;
int lock = FREE;
```

```
acquire(int* lock) {
    while (test&set(guard));
    if (*lock == BUSY) {
        put thread on wait queue
        go to sleep() & guard = FREE;
    } else {
        *lock = BUSY;
        guard = FREE;
    }
}
```

Implementing locks with test&set

- Better solution (version 4.0)

```
#define FREE 0
#define BUSY 1
int guard = FREE;
int lock = FREE;
```

```
acquire(int* lock) {
    while (test&set(guard));
    if (*lock == BUSY) {
        put thread on wait queue
        go to sleep() & guard = FREE;
    } else {
        *lock = BUSY;
        guard = FREE;
    }
}
```

```
release(int* lock) {
    while (test&set(guard));
    if (anyone on wait queue) {
        take one thread off wait queue;
        place this thread on ready queue;
    } else {
        *lock = BUSY;
    }
    guard = FREE;
}
```

Implementing locks with test&set

- Better solution (version 4.0)
  - Can be implemented by the users
  - Works well on multiprocessor
  - Has no busy-waiting

```
#define FREE 0
#define BUSY 1
int guard = FREE;
int lock = FREE;
```

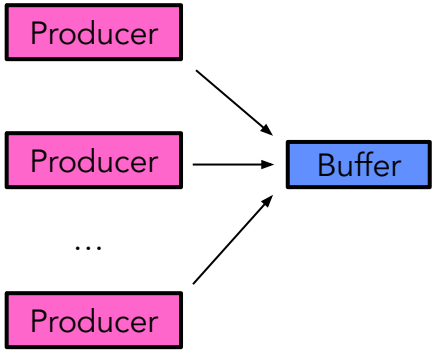
```
acquire(int* lock) {
    while (test&set(guard));
    if (*lock == BUSY) {
        put thread on wait queue
        go to sleep() & guard = FREE;
    } else {
        *lock = BUSY;
        guard = FREE;
    }
}
```

```
release(int* lock) {
    while (test&set(guard));
    if (anyone on wait queue) {
        take one thread off wait queue;
        place this thread on ready queue;
    } else {
        *lock = BUSY;
    }
    guard = FREE;
}
```

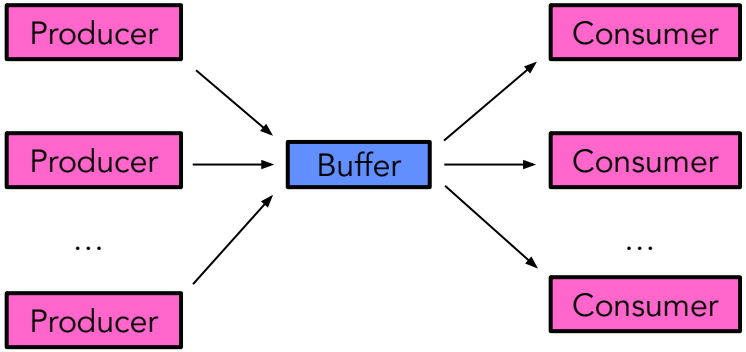
Buffer reading/writing

Buffer reading/writing

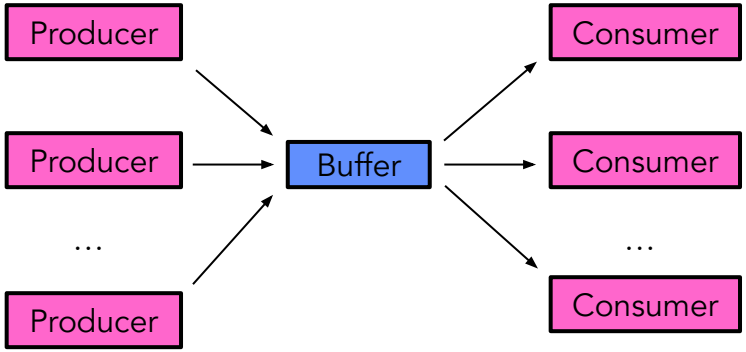
Buffer reading/writing



Buffer reading/writing

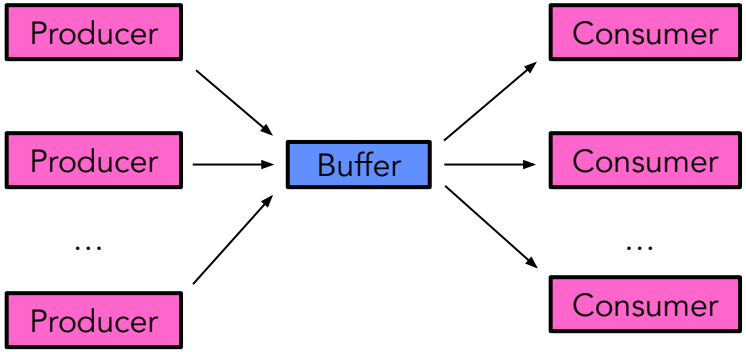


Buffer reading/writing



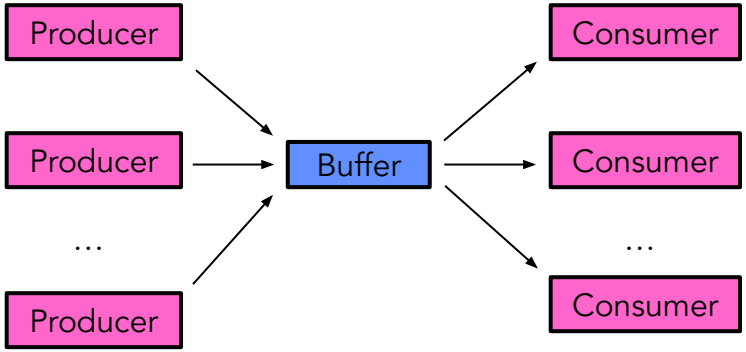
1. Only one thread can manipulate the buffer at a time

Buffer reading/writing



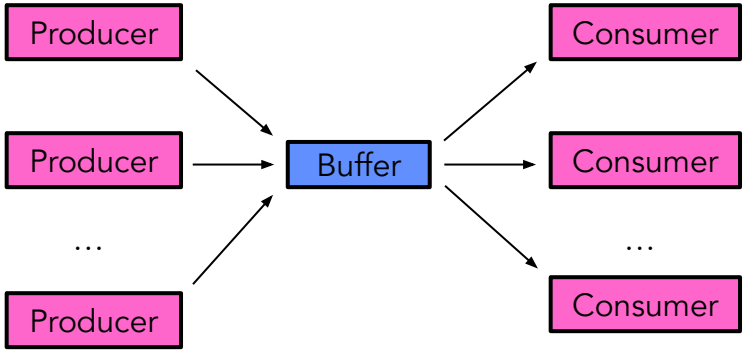
1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty

Buffer reading/writing



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

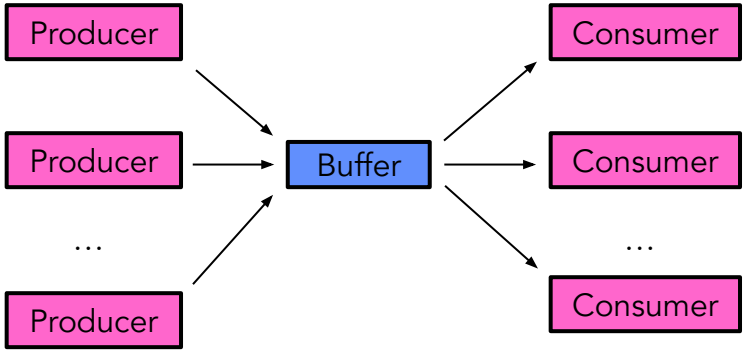
Buffer reading/writing



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

```
// Naive implementation
mutex lock = <initially unlocked>

Producer(item) {
   lock.acquire();
   while(buffer is full) {
      // wait for a free slot
   }
   enqueue(item);
   lock.release();
}

Consumer() {
   lock.acquire();
   while (buffer is empty) {
      // wait for arrival
   }
   item = dequeue();
   lock.release();
   return item;
}
```

Buffer reading/writing

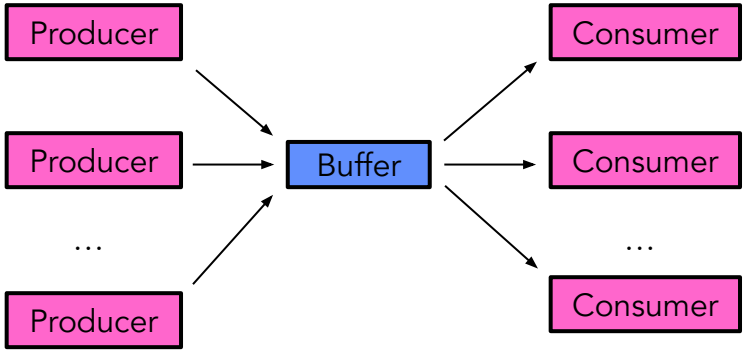


1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

```
// Naive implementation
mutex lock = <initially unlocked>

Producer(item) {
    lock.acquire();
    while(buffer is full) {
        // wait for a free slot
    }
    enqueue(item);
    lock.release();
}

Consumer() {
    lock.acquire();
    while (buffer is empty) {
        // wait for arrival
    }
    item = dequeue();
    lock.release();
    return item;
}
```

Can we jump out these loops?

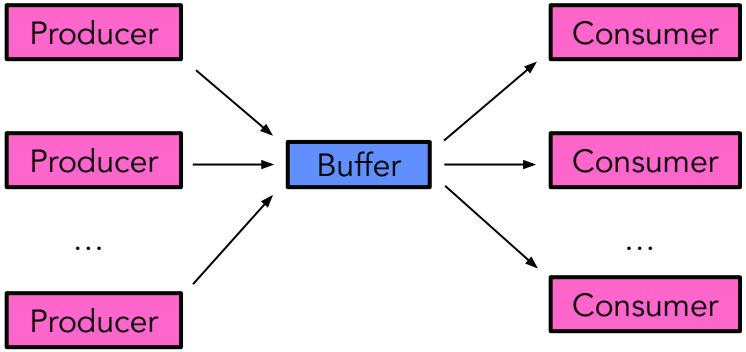
Buffer reading/writing



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

```
// Better implementation
mutex lock = <initially unlocked>

Producer(item) {
   lock.acquire();
   while(buffer is full) {
      lock.release(); lock.acquire();
   }
   enqueue(item);
   lock.release();
}

Consumer() {
   lock.acquire();
   while (buffer is empty) {
      lock.release(); lock.acquire();
   }
   item = dequeue();
   lock.release();
   return item;
}
```

Buffer reading/writing



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

```
// Better implementation
mutex lock = <initially unlocked>

Producer(item) {
   lock.acquire();
   while(buffer is full) {
      lock.release(); lock.acquire();
   }
   enqueue(item);
   lock.release();
}

Consumer() {
   acquire(&buf_lock);
   while (buffer is empty) {
      lock.release(); lock.acquire();
   }
   item = dequeue();
   lock.acquire();
   return item;
}
```

busy waiting!

Semaphores

Semaphores

- Each semaphore has a non-negative integer value and supports 3 operations

Semaphores

- Each semaphore has a non-negative integer value and supports 3 operations
  - Initialize a value
  - Down() or P()
    - An atomic operation that waits for semaphore to become positive, then decrements it by 1
  - Up() or V()
    - An atomic operation that increases the semaphore by 1, waking up a waiting P thread, if any

Semaphores

- Each semaphore has a non-negative integer value and supports 3 operations
  - Initialize a value
  - Down() or P()
    - An atomic operation that waits for semaphore to become positive, then decrements it by 1
  - Up() or V()
    - An atomic operation that increases the semaphore by 1, waking up a waiting P thread, if any
- Applications

Semaphores

- Each semaphore has a non-negative integer value and supports 3 operations
  - Initialize a value
  - Down() or P()
    - An atomic operation that waits for semaphore to become positive, then decrements it by 1
  - Up() or V()
    - An atomic operation that increases the semaphore by 1, waking up a waiting P thread, if any
- Applications
  - Initial value = 1

```
mysem = 1;
mysem.P();
// Critical section goes here
mysem.V();
```

Semaphores

- Each semaphore has a non-negative integer value and supports 3 operations
  - Initialize a value
  - Down() or P()
    - An atomic operation that waits for semaphore to become positive, then decrements it by 1
  - Up() or V()
    - An atomic operation that increases the semaphore by 1, waking up a waiting P thread, if any
- Applications
  - Initial value = 1

```
mysem = 1;
mysem.P();
// Critical section goes here
mysem.V();
```

a lock

Semaphores

- Each semaphore has a non-negative integer value and supports 3 operations
  - Initialize a value
  - Down() or P()
    - An atomic operation that waits for semaphore to become positive, then decrements it by 1
  - Up() or V()
    - An atomic operation that increases the semaphore by 1, waking up a waiting P thread, if any
- Applications
  - Initial value = 1

```
mysem = 1;
mysem.P();
// Critical section goes here
mysem.V();
```

a lock

  - Initial value = 0

```
mysem = 0;
ThreadJoin {
    mysem.P();
}
ThreadFinish {
    mysem.V();
}
```

Semaphores

- Each semaphore has a non-negative integer value and supports 3 operations
  - Initialize a value
  - Down() or P()
    - An atomic operation that waits for semaphore to become positive, then decrements it by 1
  - Up() or V()
    - An atomic operation that increases the semaphore by 1, waking up a waiting P thread, if any
- Applications
  - Initial value = 1

```
mysem = 1;
mysem.P();
// Critical section goes here
mysem.V();
```

a lock

  - Initial value = 0

```
mysem = 0;
ThreadJoin {
    mysem.P();
}
ThreadFinish {
    mysem.V();
}
```

executed before

Semaphores

- Each semaphore has a non-negative integer value and supports 3 operations
  - Initialize a value
  - Down() or P()
    - An atomic operation that waits for semaphore to become positive, then decrements it by 1
  - Up() or V()
    - An atomic operation that increases the semaphore by 1, waking up a waiting P thread, if any
- Applications
  - Initial value = 1

```
mysem = 1;
mysem.P();
// Critical section goes here
mysem.V();
```

a lock

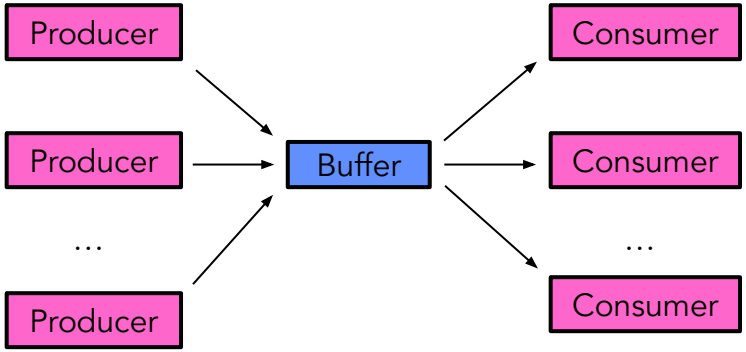
  - Initial value = 0

```
mysem = 0;
ThreadJoin {
    mysem.P();
}
ThreadFinish {
    mysem.V();
}
```

executed before

a scheduling constraint

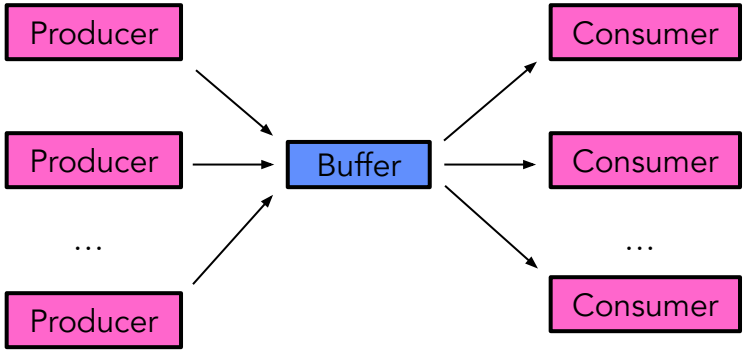
Buffer reading/writing



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

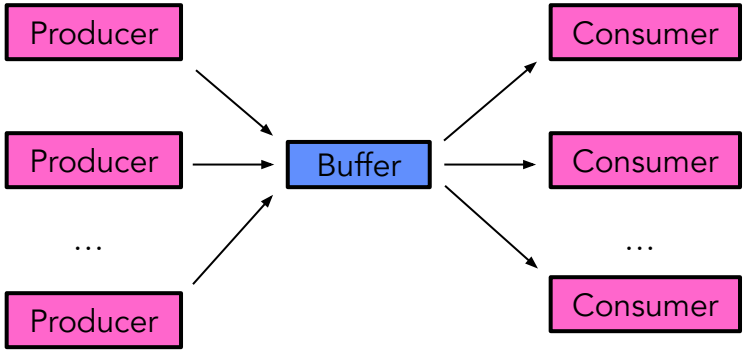
Buffer reading/writing

```
// Using semaphores
```



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

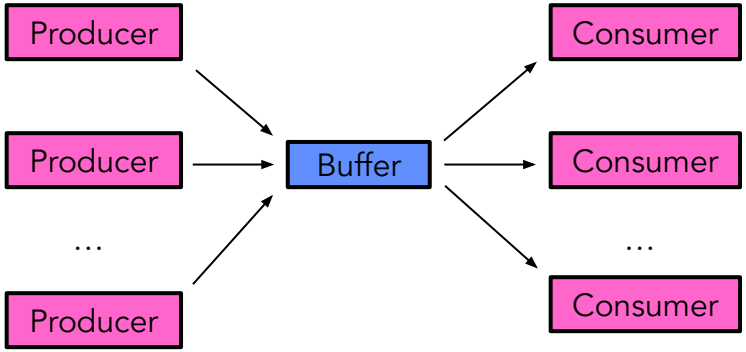
Buffer reading/writing




```
// Using semaphores
Semaphore lock = 1;

Semaphore fullSlots = 0;

Semaphore emptySlots = bufSize;
```

1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

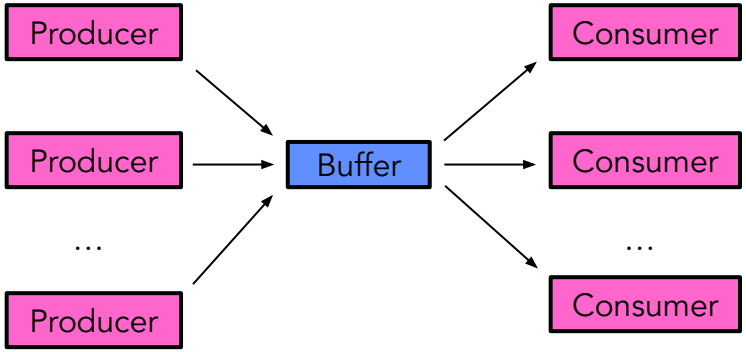
Buffer reading/writing



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

```
// Using semaphores
Semaphore lock = 1;

Semaphore fullSlots = 0;

Semaphore emptySlots = bufSize;

Producer(item) {
    emptySlots.P();
    lock.P();
    enqueue(item);
    lock.V();
    fullSlots.V();
}

Consumer() {
    fullSlots.P();
    lock.P();
    item = dequeue();
    lock.V();
    emptySlots.V();
    return item;
}
```

Buffer reading/writing

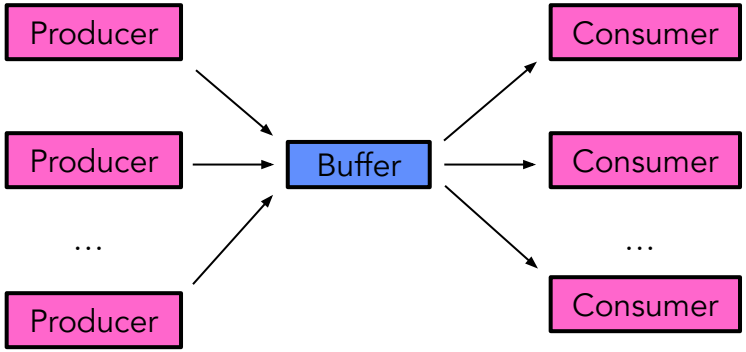


1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

all are semaphores

```
// Using semaphores
Semaphore lock = 1;

Semaphore fullSlots = 0;

Semaphore emptySlots = bufSize;

Producer(item) {
    emptySlots.P();
    lock.P();
    enqueue(item);
    lock.V();
    fullSlots.V();
}

Consumer() {
    fullSlots.P();
    lock.P();
    item = dequeue();
    lock.V();
    emptySlots.V();
    return item;
}
```

Buffer reading/writing



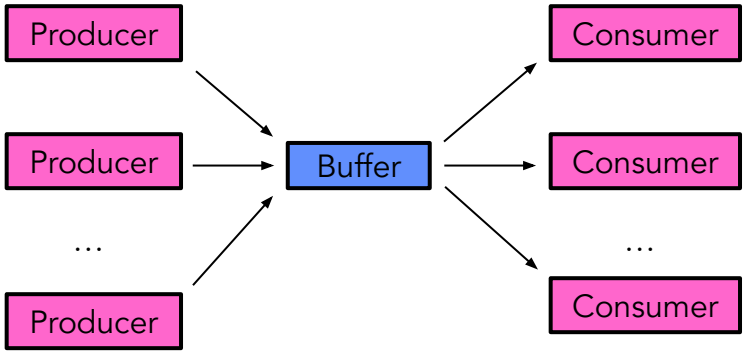
1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

all are semaphores

but here we use it as a constraint

```
// Using semaphores
Semaphore lock = 1;

Semaphore fullSlots = 0;

Semaphore emptySlots = bufSize;

Producer(item) {
    emptySlots.P();
    lock.P();
    enqueue(item);
    lock.V();
    fullSlots.V();
}

Consumer() {
    fullSlots.P();
    lock.P();
    item = dequeue();
    lock.V();
    emptySlots.V();
    return item;
}
```

Buffer reading/writing



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

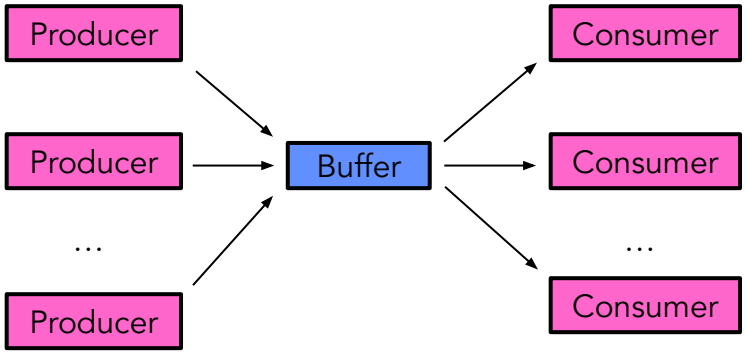
all are semaphores

but here we use it as a constraint

... and here it is a lock

```
// Using semaphores
Semaphore lock = 1;

Semaphore fullSlots = 0;

Semaphore emptySlots = bufSize;

Producer(item) {
    emptySlots.P();
    lock.P();
    enqueue(item);
    lock.V();
    fullSlots.V();
}

Consumer() {
    fullSlots.P();
    lock.P();
    item = dequeue();
    lock.V();
    emptySlots.V();
    return item;
}
```

Buffer reading/writing



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

all are semaphores

but here we use it as a constraint

... and here it is a lock

It is better to separate them explicitly!

```
// Using semaphores
Semaphore lock = 1;

Semaphore fullSlots = 0;

Semaphore emptySlots = bufSize;

Producer(item) {
    emptySlots.P();
    lock.P();
    enqueue(item);
    lock.V();
    fullSlots.V();
}

Consumer() {
    fullSlots.P();
    lock.P();
    item = dequeue();
    lock.V();
    emptySlots.V();
    return item;
}
```

Condition variables & monitors

Condition variables & monitors

- Condition Variable: a queue of threads waiting for something inside a critical section

Condition variables & monitors

- Condition Variable: a queue of threads waiting for something inside a critical section
  - Key idea: allow sleeping inside critical section by atomically releasing lock at time we go to sleep

Condition variables & monitors

- Condition Variable: a queue of threads waiting for something inside a critical section
  - Key idea: allow sleeping inside critical section by atomically releasing lock at time we go to sleep
  - Operations:
    - wait(&lock): Atomically release lock and go to sleep.
      - Re-acquire lock later, before returning.
    - signal(): Wake up one waiter, if any
    - broadcast(): Wake up all waiters

Condition variables & monitors

- Condition Variable: a queue of threads waiting for something inside a critical section
  - Key idea: allow sleeping inside critical section by atomically releasing lock at time we go to sleep
  - Operations:
    - wait(&lock): Atomically release lock and go to sleep.
      - Re-acquire lock later, before returning.
    - signal(): Wake up one waiter, if any
    - broadcast(): Wake up all waiters
  - Rule: Must hold lock when doing condition variable operations

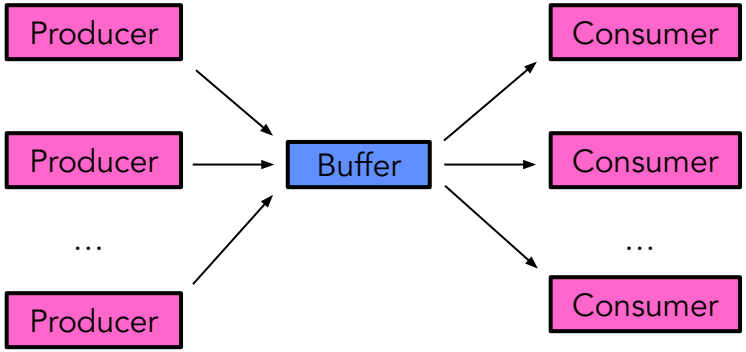
Condition variables & monitors

- Condition Variable: a queue of threads waiting for something inside a critical section
  - Key idea: allow sleeping inside critical section by atomically releasing lock at time we go to sleep
  - Operations:
    - wait(&lock): Atomically release lock and go to sleep.
      - Re-acquire lock later, before returning.
    - signal(): Wake up one waiter, if any
    - broadcast(): Wake up all waiters
  - Rule: Must hold lock when doing condition variable operations
- Monitor (version 6.0): a paradigm for concurrent programming

Condition variables & monitors

- Condition Variable: a queue of threads waiting for something inside a critical section
  - Key idea: allow sleeping inside critical section by atomically releasing lock at time we go to sleep
  - Operations:
    - wait(&lock): Atomically release lock and go to sleep.
      - Re-acquire lock later, before returning.
    - signal(): Wake up one waiter, if any
    - broadcast(): Wake up all waiters
  - Rule: Must hold lock when doing condition variable operations
- Monitor (version 6.0): a paradigm for concurrent programming
  - A monitor = a lock + one or more condition variables for managing concurrent access to shared data

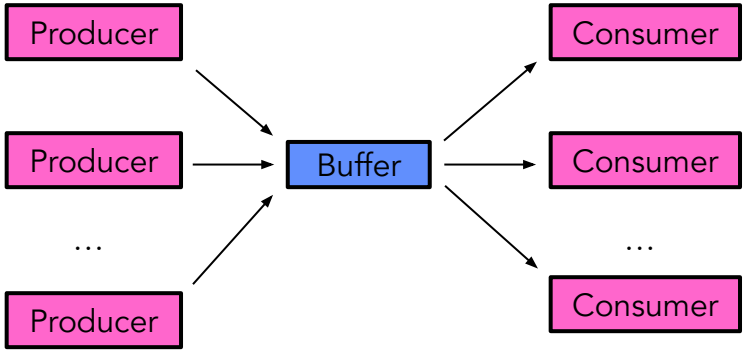
Buffer reading/writing



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

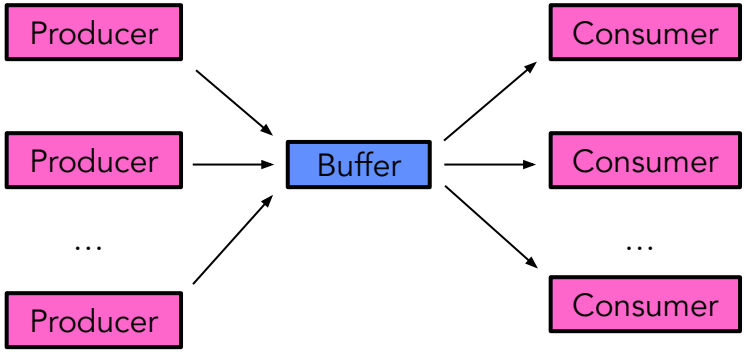
Buffer reading/writing

```
// Using monitor
```



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

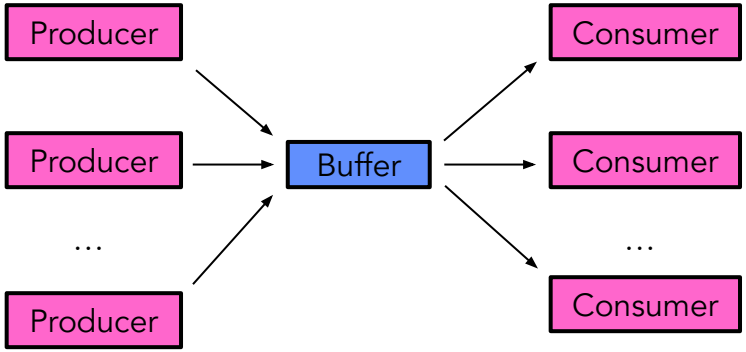
Buffer reading/writing




```
// Using monitor
lock buf_lock = <initially unlocked>
condition_variable producer_CV = <initially empty>
condition_variable consumer_CV = <initially empty>
```

1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

Buffer reading/writing



1. Only one thread can manipulate the buffer at a time
2. Consumer must wait for producer if the buffer is empty
3. Producer must wait for consumer if the buffer is full

```
// Using monitor
lock buf_lock = <initially unlocked>
condition_variable producer_CV = <initially empty>
condition_variable consumer_CV = <initially empty>

Producer(item) {
    buf_lock.acquire();
    while (buffer is full) {
        producer_CV.wait(&buf_lock);
    }
    enqueue(item);
    consumer_CV.signal();
    release(&buf_lock);
}

Consumer() {
    acquire(buf_lock);
    while (buffer is empty) {
        consumer_CV.wait(&buf_lock);
    }
    item = dequeue();
    producer_CV.signal();
    release(buf_lock);
    return item
}
```

Overview

Hardware support	Load/store	Disable interrupts	Test&set	Compare&swap

Overview

High-level APIs	Locks	Semaphores	Monitors	Send/Receive
Hardware support	Load/store	Disable interrupts	Test&set	Compare&swap

Overview

Programs	Multi-cooperating-threaded programs			
High-level APIs	Locks	Semaphores	Monitors	Send/Receive
Hardware support	Load/store	Disable interrupts	Test&set	Compare&swap

Overview

Programs	Multi-cooperating-threaded programs			
High-level APIs	Locks	Semaphores	Monitors	Send/Receive
Hardware support	Load/store	Disable interrupts	Test&set	Compare&swap

Synchronization method	Hardware support (minimum)	Work with multiprocessor	Can be implemented in user space	Busy-waiting	Be able to synchronize complex programs	Easy to synchronize complex programs
Lock (version 1.0)	Disable interrupts	No	No	“Yes”	No	–
Lock (version 2.0)	Disable interrupts	No	No	No	No	–
Lock (version 3.0, “spin lock”)	Test&set	Yes	Yes	Yes	No	–
Lock (version 4.0)	Test&set	Yes	Yes	No	Yes	No
Semaphore	Test&set	Yes	Yes	No	Yes	“No”
Monitor	Test&set	Yes	Yes	No	Yes	Yes

Language & OS support: Read-Modify-Write (in C)

NAME

stdatomic.h — atomics

SYNOPSIS

```
#include <stdatomic.h>
```

DESCRIPTION

[CX] ⌦ The functionality described on this reference page is aligned with the ISO C standard. Any conflict between the requirements described here and the ISO C standard is unintentional. This volume of POSIX.1-2024 defers to the ISO C standard. ⌫

Implementations that define the macro __STDC_NO_ATOMICS__ need not provide this header nor support any of its facilities.

The <*stdatomic.h*> header shall define the **atomic_flag** type as a structure type. This type provides the classic test-and-set functionality. It shall have two states, set and clear. Operations on an object of type **atomic_flag** shall be lock free.

The <*stdatomic.h*> header shall define each of the atomic integer types in the following table as a type that has the same representation and alignment requirements as the corresponding direct type.

Note:
The same representation and alignment requirements are meant to imply interchangeability as arguments to functions, return values from functions, and members of unions.

https://pubs.opengroup.org/onlinepubs/9799919799/basedefs/stdatomic.h.html

Language & OS support: futex

futex(2) — Linux manual page

futex(2) System Calls Manual *futex*(2)

NAME top

futex - fast user-space locking

LIBRARY top

Standard C library (*libc*, *-lc*)

SYNOPSIS top

```
#include <linux/futex.h>      /* Definition of FUTEX_* constants */
#include <sys/syscall.h>      /* Definition of SYS_* constants */
#include <unistd.h>

long syscall(SYS_futex, uint32_t *uaddr, int futex_op, uint32_t val,
             const struct timespec *timeout,   /* or: uint32_t val2 */
             uint32_t *uaddr2, uint32_t val3);
```

Note: glibc provides no wrapper for **futex**(), necessitating the use of syscall(2).

https://man7.org/linux/man-pages/man2/futex.2.html

Language & OS support: C

```
int   pthread_mutex_destroy(pthread_mutex_t *);
int   pthread_mutex_getprioceiling(const pthread_mutex_t *, int *);
int   pthread_mutex_init(pthread_mutex_t *, const pthread_mutexattr_t *);
int   pthread_mutex_lock(pthread_mutex_t *);
int   pthread_mutex_setprioceiling(pthread_mutex_t *, int, int *);
int   pthread_mutex_trylock(pthread_mutex_t *);
int   pthread_mutex_unlock(pthread_mutex_t *);
int   pthread_mutexattr_destroy(pthread_mutexattr_t *);
int   pthread_mutexattr_getprioceiling(const pthread_mutexattr_t *,
          int *);
int   pthread_mutexattr_getprotocol(const pthread_mutexattr_t *, int *);
int   pthread_mutexattr_getpshared(const pthread_mutexattr_t *, int *);
int   pthread_mutexattr_gettype(const pthread_mutexattr_t *, int *);
int   pthread_mutexattr_init(pthread_mutexattr_t *);
int   pthread_mutexattr_setprioceiling(pthread_mutexattr_t *, int);
int   pthread_mutexattr_setprotocol(pthread_mutexattr_t *, int);
int   pthread_mutexattr_setpshared(pthread_mutexattr_t *, int);
int   pthread_mutexattr_settype(pthread_mutexattr_t *, int);
```

https://pubs.opengroup.org/onlinepubs/7908799/xsh/pthread.h.html

Language & OS support: C++

Mutual exclusion

Mutual exclusion algorithms prevent multiple threads from simultaneously accessing shared resources. This prevents data races and provides support for synchronization between threads.

Defined in header <mutex>

`mutex` (C++11)	provides basic mutual exclusion facility (class)
`timed_mutex` (C++11)	provides mutual exclusion facility which implements locking with a timeout (class)
`recursive_mutex` (C++11)	provides mutual exclusion facility which can be locked recursively by the same thread (class)
`recursive_timed_mutex` (C++11)	provides mutual exclusion facility which can be locked recursively by the same thread and implements locking with a timeout (class)

Defined in header <shared_mutex>

`shared_mutex` (C++17)	provides shared mutual exclusion facility (class)
`shared_timed_mutex` (C++14)	provides shared mutual exclusion facility and implements locking with a timeout (class)

Condition variables

A condition variable is a synchronization primitive that allows multiple threads to communicate with each other. It allows some number of threads to wait (possibly with a timeout) for notification from another thread that they may proceed. A condition variable is always associated with a mutex.

Defined in header <condition_variable>

`condition_variable` (C++11)	provides a condition variable associated with a `std::unique_lock` (class)
`condition_variable_any` (C++11)	provides a condition variable associated with any lock type (class)
`notify_all_at_thread_exit` (C++11)	schedules a call to `notify_all` to be invoked when this thread is completely finished (function)

Semaphores

A semaphore is a lightweight synchronization primitive used to constrain concurrent access to a shared resource. When either would suffice, a semaphore can be more efficient than a condition variable.

Defined in header <semaphore>

`counting_semaphore` (C++20)	semaphore that models a non-negative resource count (class template)

https://en.cppreference.com/w/cpp/thread

Language & OS support: Java

ORACLE

Java SE > Java SE Specifications > Java Language Specification

Chapter 17. Threads and Locks

Prev

Table of Contents

17.1. Synchronization
17.2. Wait Sets and Notification

17.2.1. Wait
17.2.2. Notification
17.2.3. Interruptions
17.2.4. Interactions of Waits, Notification, and Interruption

17.3. Sleep and Yield
17.4. Memory Model

17.4.1. Shared Variables
17.4.2. Actions
17.4.3. Programs and Program Order
17.4.4. Synchronization Order
17.4.5. Happens-before Order
17.4.6. Executions
17.4.7. Well-Formed Executions
17.4.8. Executions and Causality Requirements
17.4.9. Observable Behavior and Nonterminating Executions

17.5. `final` Field Semantics

17.5.1. Semantics of `final` Fields
17.5.2. Reading `final` Fields During Construction
17.5.3. Subsequent Modification of `final` Fields
17.5.4. Write-Protected Fields

17.6. Word Tearing
17.7. Non-Atomic Treatment of `double` and `long`

Chapter 17. Threads and Locks

While most of the discussion in the preceding chapters is concerned only with the behavior of code as executed a single statement or expression at a time, that is, by a single *thread*, the Java Virtual Machine can support many threads of execution at once. These threads independently execute code that operates on values and objects residing in a shared main memory. Threads may be supported by having many hardware processors, by time-slicing a single hardware processor, or by time-slicing many hardware processors.

Threads are represented by the `Thread` class. The only way for a user to create a thread is to create an object of this class; each thread is associated with such an object. A thread will start when the `start()` method is invoked on the corresponding `Thread` object.

The behavior of threads, particularly when not correctly synchronized, can be confusing and counterintuitive. This chapter describes the semantics of multithreaded programs; it includes rules for which values may be seen by a read of shared memory that is updated by multiple threads. As the specification is similar to the *memory models* for different hardware architectures, these semantics are known as the *Java programming language memory model*. When no confusion can arise, we will simply refer to these rules as "the memory model".

These semantics do not prescribe how a multithreaded program should be executed. Rather, they describe the behaviors that multithreaded programs are allowed to exhibit. Any execution strategy that generates only allowed behaviors is an acceptable execution strategy.

17.1. Synchronization

The Java programming language provides multiple mechanisms for communicating between threads. The most basic of these methods is *synchronization*, which is implemented using *monitors*. Each object in Java is associated with a monitor, which a thread can *lock* or *unlock*. Only one thread at a time may hold a lock on a monitor. Any other threads attempting to lock that monitor are blocked until they can obtain a lock on that monitor. A thread *t* may lock a particular monitor multiple times; each unlock reverses the effect of one lock operation.

The `synchronized` statement (§14.19) computes a reference to an object; it then attempts to perform a lock action on that object's monitor and does not proceed further until the lock action has successfully completed. After the lock action has been performed, the body of the `synchronized` statement is executed. If execution of the body is ever completed, either normally or abruptly, an unlock action is automatically performed on that same monitor.

A `synchronized` method (§8.4.3.6) automatically performs a lock action when it is invoked; its body is not executed until the lock action has successfully completed. If the method is an instance method, it locks the monitor associated with the instance for which it was invoked (that is, the object that will be known as `this` during execution of the body of the method). If the method is `static`, it locks the monitor

https://docs.oracle.com/javase/specs/jls/se21/html/jls-17.html

Language & OS support: Python

Concurrent Execution

The modules described in this chapter provide support for concurrent execution of code. The appropriate choice of tool will depend on the task to be executed (CPU bound vs IO bound) and preferred style of development (event driven cooperative multitasking vs preemptive multitasking). Here's an overview:

- `threading` — Thread-based parallelism
  - Thread-Local Data
  - Thread Objects
  - Lock Objects
  - RLock Objects
  - Condition Objects
  - Semaphore Objects
    - `Semaphore` Example
  - Event Objects
  - Timer Objects
  - Barrier Objects
  - Using locks, conditions, and semaphores in the `with` statement

https://docs.python.org/3/library/concurrency.html

Summary

- Race condition
- Atomic operation
  - always runs to completion or not at all
- Synchronization
  - using atomic operations to ensure cooperation between threads
- Mutual exclusion
  - only one thread does a particular thing at a time
- Critical section
  - piece of code that only one thread can execute at once
- Locks
- Semaphores
- Condition variables
- Monitors
- Language & OS support